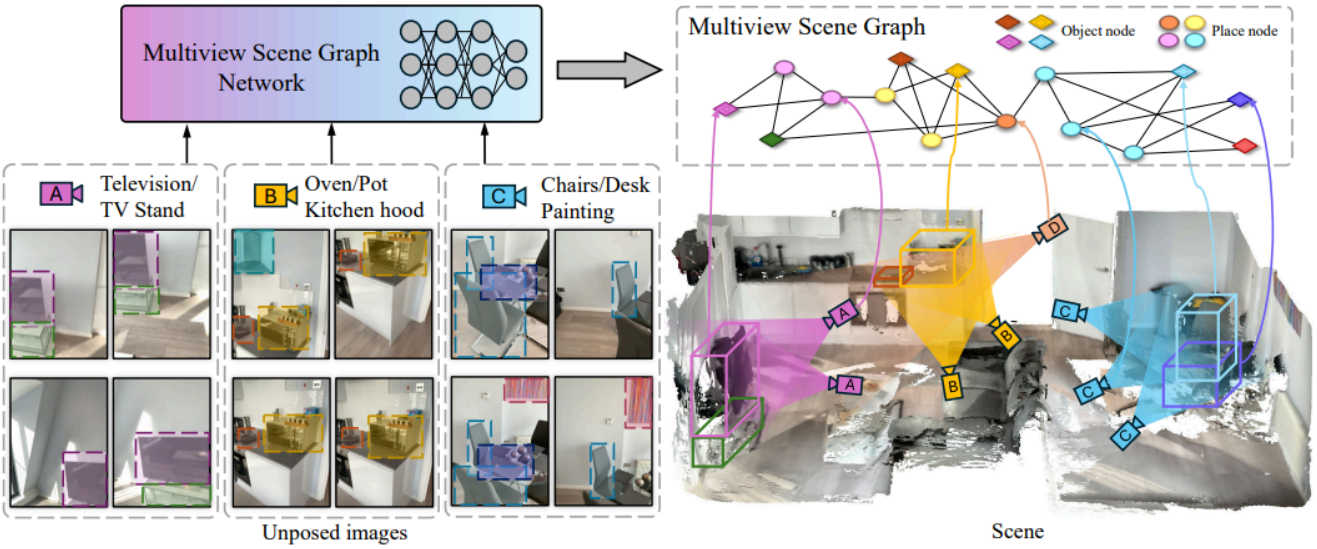
Multiview Scene Graph

> #CV #SceneMapping #HybridArchitecture



- What does this framework do?
  - From the highest level, given a set of unposed RGB images of a scene:

    $X = \{x_i\}_{i=0...T}$

    The model will return a Multiview Scene Graph as a place+object graph:

    $$G = \{P, O, E^{PP}, E^{PO}\}$$
    $$\text{where:}$$
    $$P \text{ is the place (of camera) nodes}$$
    $$O \text{ is the object nodes}$$
    $$E^{PP} \text{ is the edges between place nodes}$$
    $$E^{PO} \text{ is the edges between place and object nodes}$$
  - In practice, we present the graph by:

    $A = \begin{bmatrix} A^{PP} & A^{PO} \\ A^{OP} & A^{OO} \end{bmatrix} \in \mathbb{R}^{(|P|+|O|)\times(|P|+|O|)}$ (or $\hat{A}$ for the predicted result)
    - A^{PP} is the adjecency matrix of the place nodes (if i and j are connected, then the value of $A^{PP}[i][j] = 1$).
    - A^{PO} is the adjecency matrix of the place-object nodes
    - A^{OP} is similar, but is in the object-place order
    - A^{OO} is all 0's, as we don't care about the relationships of objects here.
  - Architecture
    - The model has hybrid architecture with transformer as the backbone.
    - Given the unposed RGB images X, we feed them to a frozen object detector (Grounding DINO) and a frozen image encoder (DINOv2).
    - The query of the transformer, Q, is from the output of object detector.
      - Normal Object:

            - The object detector will provide the feature vector of each object.
            - We normalize the coordinates of the bounding boxes of the objects and project the normalized coordinates to obtain an embedding, which is the **positional encoding**.
            - Element-wisely add the bounding box encoding to the feature vector of each object.
        - Place
            - **We regard each place as a huge object to unite the method of handling places and objects.**
            - The bounding box of this object covers the whole image, so after normalization, the coordinates are always $[0, 0, 1, 1]$ (top-left, bottom-right corner).
            - Embedding the bounding box and element-wise addition.
    - The key and value of the transformer comes from the image encoder.
    - We run both self-attention (among Q) and cross attention (among Q, K, V) to update the representation of places and objects.
    - Output O_{obj} and O_{place}: each detected object has a vector that represents it; each image has a vector that represents its place.
- Losses: Supervised Contrasive Learning
    - For place: MSE on Cosine Distance

      $L_{place} = \mathrm{MSE}(\mathrm{CosineDistance}(e_i, e_j), \mathrm{Label}_{i,j})$
        - Given two images, we want to minimize the difference between the cosine distance and the actual distance of the position of the camera.
        - If i, j are taken at the same spot (distance < 1m and rotation < 1 rad), and the cosine distance is large, the loss will be high, and vice versa.
    - For objects: Binary Cross-Entropy

      $L_{object} = \mathrm{Weighted_BCE}(\mathrm{Similarity}(o_i, o_j), \mathrm{Match}_{i,j})$
        - In most cases, each pair of objects is not the same object; the actual matched (but from different camera position) objects are sparse.
        - A higher weight is added to the situation where two objects are the same but from different pov. That is to say, if the model *classifies* this pair as "not the same object", it will have 10x penalties.
- Inference and graph construction:
    - Place: With O_{place}, if the cosine similarity between a pair is greater than a threshold, then this pair is considered the same position, and a P-P edge is created.
    - Object: a memory bank is maintained. For each object in a new frame, use O_{obj} to calculate cosine similarity with those that have already been added to the memory. If the distance < a threshold, merge the objects. Otherwise, create a new node, and connect it to the place node of that frame.
- Evaluation
    - Place-place evaluation: directly use IoU

$$PP\ IoU = IoU(A^{PP}, \hat{A}^{PP}) = \frac{\text{TP}}{\text{TP+FP+FN}},\ \text{where P} = 1.$$

- Place-Object IoU:
  - Hard because the number of objects predicted by the model (the size of $\hat{O}$) may differ from the actual number of objects (the size of O), and the order is random.
  - Use Accumulated GIoU -> Hungarian Match (to align the result):

    $$PO\ IoU = IoU(A^{PO}, \hat{A}^{PO}S)$$

- Experiment Results
  - Key Comparisons:
    - AoMSG vs. SepMSG: AoMSG outperforms baselines that separately handle location recognition and object association (SepMSG-Direct/Linear/MLP). For instance, AoMSG-4 achieves 74.2% PO IoU, while the linear baseline only reaches 59.3%.
    - AoMSG vs. Traditional Methods: For object association, AoMSG significantly outperforms Unitrack (13.0 PO IoU) and DEVA (16.6 PO IoU). This indicates traditional tracking methods struggle with large viewpoint changes or long-duration object disappearances and reappearances.
  - Qualitative Analysis:
    - Visualizations in embedding space reveal that AoMSG pulls embeddings of the same object closer together across different viewpoints while pushing distinct objects farther apart, achieving superior separation compared to pre-trained features relying solely on linear probes.
  - Application Demonstration:
    - The paper demonstrates using MSG to assist Dust3r in local 3D reconstruction. By leveraging the topological subgraphs provided by MSG, efficient, object-centered local reconstruction is achievable without processing massive image volumes.
- My Thoughts
  - The architecture is very promising. However, one drawback is the lack of human-interaction ability.
  - We can, via feature alignment, make the architecture a Language-Guided MSG. That is to say, align the Object Embedding Space with the embedding space of the natural language embedding.
  - We can use a **frozen** CLiP model as the teacher, and the MSG as the student.
  - Given the output of object O_{obj}:
    - Plan 1: distillation with MSE / Cosine Embedding Loss:
      - Use the output of the object detector to feed to the CLiP and get the CLiP output embedding E_{clip_img}
      - Let the align loss be:

        $$L_{align} = 1 - \text{CosineSimilarity}(Proj(O_{obj}), E_{clip_img})$$

        Prj is the matrix to make the dimension aligned.
    - Plan 2: Contrastive Distillation (text-supervised)

      (If the text labels of the input images are provided)
      - Use CLiP NL-encoder to get the embedding of the labels: E_{text}

        - Calculate the similarity between O_{obj} and E_{text}.
        - Using InfoNCE Loss or Cross Entropy, force O_{obj} to match its corresponding category text E_{text}.
    - Inference time:
        - User input: "Find the place with a messy desk."
        - Processing flow:
            1. Input text "messy desk" into CLIP Text Encoder $\rightarrow$ obtain vector V_{query}.
            2. Iterate through all object nodes $\{O_1, O_2, \dots\}$ in MSG.
            3. Compute $Sim(O_i, V_{query})$.
            4. Locate the object node O_{target} with the highest similarity score.
            5. Directly index to the connected location node P_{target} via the graph structure E^{PO}.
            6. Result: The robot navigates directly to that location.
    - Final Objective Function:

      $L_{total} = L_{place} + L_{object} + \lambda \cdot L_{align}$
- Work Cited
    - Paper
    - NotebookLM
    - Chat History